Security Assessment Report

Cozy Router Euler Helpers

March 2026

Prepared for Cozy Finance

Table of Contents

Project Summary 3
Project Scope 3
Project Overview 3
Protocol Overview 3
Assessment Methodology 4
Threat and Security Overview 5
Assets 5
Actors 5
Trust Assumptions 6
Attack Vectors 6
Findings Summary 7
Severity Matrix 7
Detailed Findings 8
Low Severity Issues 9
L-01: Multiple Euler helper flows do not preserve EVault type(uint256).max full-balance semantics 9
Informational Severity Issues 11
I-01: Cozy Router supply helpers incompatible with ERC4626EVCCollateralSecuritize vaults 11
I-02: Permit2 transfer failure prevents ERC20 fallback 12
Disclaimer 14
About Certora 14

Project Summary

Project Scope

Project Name	Repository Link	Initial Commit Hash	Final Commit Hash	Platform
Cozy Router Euler Helpers	cozy-safety-module-router	d8ef27c	dba504f	EVM/Solidity

Project Overview

This document describes the security review of **Cozy Router Euler Helpers**. The work was undertaken from **February 25th, 2026** to **March 3rd, 2026**.

The following contract list is included in our scope:

- src/lib/router/EulerVaultHelpers.sol
- src/lib/router/TokenHelpers.sol

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Cozy Finance is a modular onchain **protection system** where protocols create safety modules that pool capital to cover users when predefined loss events occur. The **Cozy Router** is the **execution layer** that lets users run multi-step Cozy actions in one call and handles token movement/approvals across modules. **EulerVaultHelpers** is the **Router integration layer** for **Euler Vaults**, exposing supply, withdraw, borrow, repay, and combined convenience flows. It maps Router actions to the correct **Euler Vault / EVC call patterns**, including account-context handling for subaccounts and batched execution. It also includes **legacy** direct vault paths and newer **EVC-batched paths**, so integrations can use either simple direct flows or full EVC-native workflows.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The threat model below covers **EulerVaultHelpers** and **TokenHelpers** in the **Cozy Router**, with focus on **Router-EVC-EVault** integration boundaries. It combines the presented audit **findings** with additional **attack vectors** identified during adversarial and edge case review. The objective is to map assets, actors, trust assumptions, and the ways safety or liveness can break under misconfiguration, misuse, or other edge case conditions.

Assets

- **User underlying ERC20** balances moved through the router into/out of Euler vaults.
- **Euler vault share tokens** (including collateral-share balances held by EVC accounts/subaccounts).
- **Borrowed liabilities** and **account health** state in Euler positions.
- **Approval rights**: ERC20 allowances, Permit2 allowances, signed permits, and permit nonces.
- **WETH/ETH** balances in wrap/unwrap helper paths.
- **Safety Module** / **Rewards Manager** stake assets and receipt tokens in combined flows.
- **EVC account-control state**: enabled collateral, enabled controllers, operator authorizations, lockdown/permit-disabled flags.

Actors

- **End users** (EOAs or smart accounts) and their **EVC subaccounts**.
- **EVC account operators** authorized via setOperator / setAccountOperator.
- **Cozy Router** (execution/aggregation coordinator).
- **Euler EVC** (context/authentication and status-check engine).
- **Euler vault contracts** (liability vaults, collateral vaults, restricted periphery vaults).
- **Permit2** contract and any relayer/integrator submitting signed approvals.
- **RewardsManager** and **SafetyModule** contracts used in composed flows.
- Adversarial **callers/integrators** crafting malicious or griefing call bundles.

Trust Assumptions

- **Euler vault instances** passed into helpers match the intended capability model (collateral-only vs liability/borrow-enabled).
- **Permit2/ERC20 approval** state used for pulls is intentional and correctly configured by users/integrators.
- **External tokens** behave within integration expectations.
- Router **custody invariants** are sufficient (receiver checks, address-prefix family checks, no unintended router-held balances).
- **EVC auth/context** is correct (**onBehalfOfAccount** + operator scope cannot be spoofed)

Attack Vectors

- **Permit-path** liveness attack: Permit2 branch is chosen, Permit2 transfer reverts, and whole batch fails even when direct ERC20 transfer could have worked.
- **Operator-scope escalation** attempts via crafted aggregate/EVC call targeting unintended subaccounts.
- Vault capability mismatch **griefing** (e.g., collateral-only vault used in borrow/repay path) to force deterministic reverts.
- **Transfer-restriction censorship** from specialized vault shares that block router-mediated intermediate transfers.
- **Max-amount semantic mismatch** (helper pre-pull vs vault "max means full balance") causing unavoidable revert paths.
- **Non-standard** token behavior (fee-on-transfer/rebasing/blacklist/pause) causing amount drift or selective denial of execution.
- **EVC lockdown** mode abuse to block execution.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	1	1	1
Informational	2	2	0
Total	3	3	1

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
L-01	Multiple Euler helper flows do not preserve EVault type(uint256).max full-balance semantics	Low	Fixed
I-01	Cozy Router supply helpers incompatible with ERC4626EVCCollateralSecuritize vaults	Informational	Acknowledged
I-02	Permit2 transfer failure prevents ERC20 fallback	Informational	Acknowledged

Low Severity Issues

L-01: Multiple Euler helper flows do not preserve EVault type(uint256).max full-balance semantics

Severity: Low	Impact: Low	Likelihood: Low
Files: EulerVaultHelpers.sol	Status: Fixed	

Description:

Some Euler helper flows are not compatible with EVault's type(uint256).max "use full balance" behavior. In EVault, deposit(type(uint256).max, receiver) and redeem(type(uint256).max, receiver, owner) are interpreted as "use full balance" flows.

```
function deposit(uint256 amount, address receiver) public virtual nonReentrant
returns (uint256) {
    // ...
    Assets assets = amount == type(uint256).max ?
vaultCache.asset.balanceOf(account).toAssets() : amount.toAssets();
    if (assets.isZero()) return 0;
    /// ...
}

function redeem(uint256 amount, address receiver, address owner) public virtual
nonReentrant returns (uint256) {
    /// ...
    Shares shares = amount == type(uint256).max ?
vaultStorage.users[owner].getBalance() : amount.toShares();
    if (shares.isZero()) return 0;
    /// ...
}
```

However, in supplyToEulerVault, supplyToEulerVaultAndStake, and supplyToEulerVaultDepositAndStake, _pullTokenFromMsgSender executes before eVault.deposit, so passing type(uint256).max attempts to transfer that exact amount (impossible) instead of

letting EVault resolve full balance. On the redeem side, redeemFromEulerVault and redeemFromEulerVaultAndUnwrap accept uint160, so type(uint256).max cannot be forwarded at all. This is an integration limitation in these helpers and can make CozyRouter::aggregate(calls[]) revert when a batched call relies on max-amount behavior, resulting in temporary execution DoS.

Recommendations:

Update these helpers to explicitly support max-amount behavior by resolving type(uint256).max to the caller's actual balance before pulling, and, also, align redeem inputs to uint256.

Customer Response:

Fixed in commit dba504f .

Fix Review:

Fixed.

Informational Severity Issues

I-01: Cozy Router supply helpers incompatible with ERC4626EVCCollateralSecuritize vaults

Files:

src/lib/router/EulerVaultHelpers.sol

Description:

Cozy's EulerVaultHelpers includes "direct" EVault supply helpers that call IEVault.deposit without going through the EVC batching flow:

- `supplyToEulerVault()`
- `supplyToEulerVaultDepositAndStake()`
- `supplyToEulerVaultAndStake()`

These helpers assume a generic Euler EVault, but are incompatible with the restricted Securitize vault ERC4626EVCCollateralSecuritize . In this vault, deposit() has been overridden to enforce:

```
  /// @notice Deposits a certain amount of assets for a receiver.
  /// @dev Only allows deposits to an account that belongs to the message sender.
  // ...
  /// @dev If called directly (not through EVC batch), the sender's owner address
needs to be registered in EVC
  function deposit(uint256 assets, address receiver)
      public
      virtual
      override
      callThroughEVC
      // ...
  {
      if (!isCommonOwner(_msgSender(), receiver)) revert NotAuthorized();
      // ...
  }
```

However, in the Cozy Euler helpers, the router calls:

```
vaultTokenAmount_ = eVault_.deposit(assetAmount_, receiver_);
```

Because of callThroughEVC, the EVC ends up calling the vault on behalf of the router contract, so inside the vault, the _msgSender() resolves to the router (not the end user). isCommonOwner(_msgSender(), receiver_) becomes isCommonOwner(router, receiver_), which fails because the router is not an EVC account with a registered owner. This is an incompatibility issue resulting in these special Securitize vaults to not be able to work with the Cozy router, as it will always revert with NotAuthorized on deposit, and cannot be used to supply assets.

Recommendations:

It is recommended to document that the direct supply helpers (supplyToEulerVault, supplyToEulerVaultAndStake, supplyToEulerVaultDepositAndStake) are not compatible with restricted EVC collateral vaults like ERC4626EVCCollateralSecuritize.

Customer Response:

Acknowledged.

Fix Review:

Acknowledged.

I-02: Permit2 transfer failure prevents ERC20 fallback

Files:

src/lib/router/EulerVaultHelpers.sol

Description:

The EulerVaultHelpers::_pullTokenFromMsgSender is intended to pull user tokens through Permit2 when possible, and otherwise use normal ERC20 transferFrom. However, once the Permit2 pre-check passes, it directly executes Permit2.transferFrom without fallback-on-failure behavior. If the Permit2 transfer reverts, the transaction reverts immediately, even when direct ERC20 approval to the router exists and ERC20 transfer would succeed.

```
/// @dev Pulls `amount_` of `token_` from msg.sender to this router. Uses Permit2
when allowance is valid, otherwise
/// falls back to ERC20 transferFrom.
function _pullTokenFromMsgSender(address token_, uint256 amount_) internal {
(uint160 permit2Allowance_, uint48 permit2Expiration_,) =
  Permit2Lib.PERMIT2.allowance(msg.sender, token_, address(this));
if (amount_ <= type(uint160).max && permit2Allowance_ >= amount_ &&
uint256(permit2Expiration_) >= block.timestamp) Permit2Lib.PERMIT2
.transferFrom(msg.sender, address(this), uint160(amount_), token_);
else IERC20(token_).safeTransferFrom(msg.sender, address(this), amount_);
}
```

A common case is when the user has configured Permit2 allowance for the router, but has not approved the token to Permit2 (or revoked that approval later). The helper selects the Permit2 path and calls Permit2.transferFrom, which reverts. Because this revert is not handled, execution never reaches the ERC20 fallback path, even though ERC20.transferFrom(user, router, amount) would succeed with the user's direct router approval. Other cases include token-level restrictions on Permit2 path (blacklist, compliance, hook checks).

Recommendations:

Wrap Permit2 transfer in try/catch or low-level call, and on failure fallback to IERC20(token).safeTransferFrom(msg.sender, address(this), amount_).

Customer Response:

Acknowledged.

Fix Review:

Acknowledged.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.